

ROS2 입문 · 5일 집중과정 강의계획서

Ubuntu 24.04 · ROS2 Jazzy · Gazebo Harmonic

과정 개요

- 대상: Python 함수·클래스와 기본 터미널 사용이 가능한 대학생·개발자. 처음 배우는 사람은 무료 샘플과 Python 기초를 먼저 학습합니다.
- 운영안: 5일, 하루 순수 수업 6시간, 총 30시간. 오전 3시간 + 오후 3시간이며 점심·휴식은 제외합니다. 예시 운영 시간은 09:00~12:00, 13:00~16:00이고 휴식이 필요하면 종료 시간을 조정합니다.
- 범위: ROS2 통신에서 모델링·시뮬레이션까지 핵심 흐름을 실습합니다. 15주 과정의 모든 과제를 압축 수행하는 과정은 아니며, Action 취소·실패와 복잡한 비동기 설계는 강사 시연·코드 분석 중심입니다.
- 준비: 개강 전 별도 3~6시간의 설치·예습 시간을 확보합니다. 설치 시간은 30시간에 포함하지 않으며 환경에 따라 늘어날 수 있습니다.

학습 성과와 수료 기준

1. 공개 예제를 빌드하고 Topic·Service·Action의 차이를 설명한다.
2. Launch·YAML·Namespace를 수정하고 결과를 로그로 확인한다.
3. TF·URDF·Xacro 모델을 읽고 RViz2에서 주요 연결을 점검한다.
4. 센서·제어 Topic을 추적하고 작은 통합 과제를 재현 가능하게 제출한다.

수료 운영안은 출석 80% 이상, 일별 실습 확인 5회 중 4회 이상, 미니 프로젝트 제출입니다. 평가 비중은 일별 실습 50%, 팀 프로젝트 30%, 개인 설명·회고 20%로 총 100%입니다. 수료 인정 기준은 교육기관이 개강 전에 확정합니다.

개강 전 필수 점검

강사는 개강 3일 전까지 설치 안내를 배포하고 전날 결과를 확인합니다. 수강생은 README Quick Start를 마쳐 /message 수신 로그를 제출하고 Turtlesim·RViz2 창, Gazebo 월드·로봇 생성 여부를 점검합니다. 미완료자는 사전 설치 클리닉 또는 준비된 실습 PC를 이용합니다. 기본 과정에는 실제 로봇·카메라·AI 모델이 필요하지 않습니다.

1일차 · 개발 환경과 Topic 첫 실행

교재 범위: 1~10장 중심 · 수업 6시간

오전 · 3시간

환경 점검·ROS 그래프 1시간 / Workspace·Node 1시간 / Topic·클래스형 Node 1시간

오후 · 3시간

Publisher·Subscriber 수정 1.5시간 / Launch 통합 1시간 / 확인·기록 0.5시간

실습 코드와 완료 기준

- 코드: maker_basic/simple_node, message_pub, message_sub, class_message.launch.py
- 완료 기준: 메시지 내용을 바꿔 송수신하고 새 터미널에서 Launch로 재현한다. /message를 Type 명시 echo로 읽는다.
- 제출물: 환경 확인표, 수정 코드, 송수신 로그.

2일차 · 통신 설계와 실행 설정

교재 범위: 11~16장 · 수업 6시간

오전 · 3시간

DDS·QoS 비교 1시간 / UserInt Message 1시간 / AddAndOdd Service 1시간

오후 · 3시간

Executor·비동기 개념 및 로그 비교 1시간 / Parameter·YAML 1시간 / 통합 실습·점검 1시간

실습 코드와 완료 기준

- 코드: maker_basic의 qos_message_pub, qos_message_sub, user_int_pub, user_int_sub, add_server, add_client, threaded_add_server, status.launch.py
- 완료 기준: QoS 조건별 수신 여부, Service 입력·응답, YAML 변경 전후 상태를 기록한다.
- 제출물: 통신 선택표, YAML 파일, 요청·응답 검증표.

3일차 · Action·Namespace·TF

교재 범위: 17~22장 · 수업 6시간

오전 · 3시간

Action 성공 흐름 1.5시간 / Feedback·Cancel·Abort 코드 추적 1시간 / 통신 방식 선택 0.5시간

오후 · 3시간

두 Turtlesim Namespace 1시간 / TF Broadcaster·Listener 1.5시간 / 팀 과제 선정 0.5시간

실습 코드와 완료 기준

- 코드: gong_basic/action_server, action_client 8, two_turtle.launch.py, mv_turtle_ns; tf2_basic/tf_listener 및 Broadcaster 예제
- 완료 기준: Goal·Feedback·Result를 구분하고 로봇별 Topic 분리를 확인한다. TF 출력의 기준 좌표계를 설명한다. 취소·실패는 시연 또는 코드 추적 결과와 실제 실행 여부를 구분한다.
- 제출물: 상태 전이도, TF 그림, 미니 프로젝트 요구 2개.

4일차 · 로봇 모델과 시각화

교재 범위: 23~26장 · 수업 6시간

오전 · 3시간

URDF Link-Joint 1시간 / 관절 운동 1시간 / RViz2 점검 1시간

오후 · 3시간

Xacro·접두어 1시간 / 모델 확장 또는 팀 과제 구현 1.5시간 / 동료 확인 0.5시간

실습 코드와 완료 기준

- 코드: tf2_basic의 two_link_fixed.urdf, two_link.urdf, ch24_joint_motion.launch.py, prefixed_arm.urdf.xacro, ch25_two_arms.launch.py
- 완료 기준: /joint_states와 팔 회전을 대조하고 두 모델의 TF 이름 충돌을 확인한다. GUI 발행자와 코드 발행자는 동시에 실행하지 않는다.
- 제출물: 모델 변경본, RViz2 화면 또는 미검증 항목 기록.

5일차 · 시뮬레이션과 통합 시연

교재 범위: 27장·종합 · 수업 6시간

오전 · 3시간

Gazebo 생성·Bridge 확인 1시간 / 벽 따라가기 관찰 1시간 / 합성 입력 검사와 결과 비교 1시간

오후 · 3시간

미니 프로젝트 마무리 1.5시간 / 팀 시연 1시간 / 개인 평가·회고 0.5시간

실습 코드와 완료 기준

- 코드: wall_follow_lab/wall_room.launch.py, wall_follow, tests/test_controller.py
- 완료 기준: /clock-/scan-/cmd_vel Type과 흐름을 확인하고 제어 반응을 기록한다. 다른 수강생이 프로젝트 절차를 따라 실행한다.
- 제출물: 코드, 실행 README, 요구별 검증 증거, 개인 회고.

미니 프로젝트와 후속 학습

2~3인 팀으로 3일차에 요구 두 개를 정하고 4~5일차의 배정된 시간 안에 구현합니다. 다중 Turtlesim 명령 분리, Xacro 팔 모델 변경, 벽 따라가기 조건 비교 중 하나를 선택합니다. 처음부터 새 시스템을 만들기보다 제공 예제에서 한 가지 변경을 완성하고 증거를 남깁니다.

프로젝트 30점은 요구 충족 15점, 재현 절차 10점, 실패 조건 기록 5점으로 평가합니다. 개인 설명·회고 20점은 코드·통신 흐름 설명 10점과 본인 수정·검증 근거 10점으로 평가합니다. 팀 시연은 팀당 5분을 기준으로 하고 12팀을 초과하면 동시 시연으로 전환합니다.

과정 후에는 15주 계획의 7~9주 비동기·Action 시험을 보강하고, 11~14주 모델링과 제어를 반복합니다. 19장의 취소·실패 시나리오와 20장의 종합 과제는 별도 자율 과제로 확장합니다.

실습 준비와 운영

- 환경: Ubuntu 24.04, ROS2 Jazzy, Python 3, colcon, Git, VS Code. Windows 사용자는 WSL2와 WSLg를 먼저 점검합니다. 후반 실습에는 RViz2와 Gazebo Harmonic이 필요합니다.
- 준비 순서: [무료 샘플 2~3장](#)의 OS-ROS2 설치 → [README Quick Start](#)의 저장소 복제·rosdep-4개 패키지 빌드 → class_message 통신 확인 → [27장 안내](#)의 Gazebo 의존성 설치·wall_follow_lab 별도 빌드 순서로 진행합니다.
- 모든 실행은 Ubuntu Bash에서 진행합니다. 새 터미널마다 source /opt/ros/jazzy/setup.bash와 source ~/[ros_ws/install/setup.bash](#)를 실행합니다. 작업 공간을 다른 경로에 만들었다면 경로를 바꿉니다.
- 강사는 개강 전 학생과 동일한 PC 환경에서 Turtlesim, RViz2, Gazebo GUI와 /clock:/scan:/cmd_vel 연결까지 예행연습합니다. 여러 학생이 같은 네트워크를 쓰면 실습별 ROS_DOMAIN_ID를 배정하고 한 학생의 터미널끼리는 동일하게 맞춥니다.
- 준비물: 개인 개발 PC, 인터넷, 코드 편집기, 실행 로그 저장 공간. 시뮬레이션 중심으로 운영하며 실제 로봇은 필수가 아닙니다. 장비 성능은 실제 수업 월드 실행으로 판단합니다.

제출과 문제 해결 원칙

매 실습은 수정한 코드 또는 Git diff, 실행 명령, 예상 결과와 실제 결과, 로그 또는 화면, 실패 원인과 수정 내용을 제출합니다. 재현 절차에는 패키지·실행 항목·파라미터를 적습니다. AI를 사용했다면 질문 요약, 채택한 코드와 직접 확인한 결과를 함께 기록합니다.

Package not found는 빌드와 source, 통신 무응답은 Node·Topic·Type·ROS_DOMAIN_ID·QoS 순서로 확인합니다. RViz2 모델 누락은 Fixed Frame·Robot Description·TF·/joint_states를 확인합니다. Gazebo 센서 누락은 재생 상태·Bridge:/scan을 확인하고, 예상과 다르게 움직이면 Pause 후 원인을 찾습니다.

자료와 검증 범위

- 주교재: 최수길, 『ROS2 입문』, 마담. [책 소개와 전체 목차](#)
- 공개 코드: [장별 코드 대응표](#), [실습 저장소](#)
- 공개 PDF는 1~3장 샘플이며 전체 교재는 포함되지 않습니다. 이 강의계획서는 수업 운영을 위한 별도 문서입니다.
- [저장소 검증 기록](#)은 2026-09-08 기준입니다. 이 계획서 작성 시 모든 Service·Action 시나리오, RViz2 GUI, Gazebo 물리 주행을 새로 검증한 것은 아닙니다. 합성 입력 검사 성공은 실제 주행 성공과 구분합니다.
- GUI 실습이 막힌 경우 Topic 로그·TF 또는 URDF 구조·합성 입력 검사를 대체 증거로 제출하고, GUI·주행 완료 여부는 미검증으로 남깁니다. 강사는 보충 실습 기회를 제공합니다.